

ДИСЦИПЛИНА	Технологии разработки серверных приложений
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям
ПРЕПОДАВАТЕЛЬ	Дворецкий Артур Геннадьевич
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Ссылка на материал:

https://github.com/dvOretsky/fastapi-tutorial/blob/main/FAPI7_Roles/FAPI7_Roles.md

Практическое занятие №7: Управление доступом на основе ролей

Управление доступом на основе ролей (RBAC - Role Based Access Control) — это подход, который используется для управления правами доступа в приложениях. В основе **RBAC** лежит разделение пользователей на группы, или роли, с соответствующими правами.

Как это работает? Представь, что твое веб-приложение — это большой офис, в котором есть разные сотрудники. У каждого сотрудника есть своя роль: кто-то работает в бухгалтерии, кто-то в отделе продаж, а кто-то в техническом отделе. У каждой роли есть доступ к определенным ресурсам. Например, бухгалтеру разрешено видеть и изменять финансовые данные, а сотруднику отдела продаж — только данные клиентов.

Важное преимущество RBAC — это централизованное управление правами доступа. Ты можешь назначить роль пользователю и предоставить доступ к нужным ресурсам, не заботясь о том, кто конкретно этот пользователь, а просто полагаясь на его роль в системе.

Использование **RBAC** помогает избежать несанкционированного доступа и упрощает управление правами доступа.

Компоненты RBAC

RBAC состоит из трех основных компонентов, которые помогают организовать и управлять доступом в приложении:

- **Роли** — это категории, которые определяют, какие действия доступны пользователю. Роль может быть связана с обязанностями в организации или с функциональностью приложения. Например, роль "администратор" может включать полный доступ ко всем функциям, а роль "пользователь" — только доступ к просмотру данных.
- **Разрешения** — это конкретные действия или операции, которые пользователь может выполнять в приложении. Разрешения могут быть такими, как "чтение", "запись", "удаление" и

т.д. Например, разрешение "чтение" позволяет просматривать информацию, а разрешение "запись" — вносить изменения в данные.

- **Пользователи** — это люди, которым назначаются роли. Один пользователь может иметь одну или несколько ролей, в зависимости от того, какой доступ ему необходим для выполнения своих задач. Например, сотрудник, который работает как администратор и пользователь, может иметь обе роли, предоставляющие различные уровни доступа.

Эти компоненты вместе обеспечивают гибкую систему управления доступом, минимизируя риск несанкционированного доступа.

Внедрение RBAC в FastAPI

Для реализации **RBAC** в **FastAPI** используется сочетание аутентификации (например, **JWT**) и авторизации, основанной на ролях. Вот обзор шагов по внедрению **RBAC**:

- **Шаг 1: Определите роли и разрешения.** Прежде чем внедрять **RBAC**, нужно определить, какие роли будут доступны в вашем приложении. Например, можно использовать такие роли, как "администратор", "пользователь" и "гость". Каждая роль будет иметь набор разрешений, которые определяют, что пользователь с этой ролью может делать в системе.
- **Шаг 2: Свяжите роли с пользователями.** После определения ролей, необходимо назначить их пользователям. Это можно сделать во время регистрации, при добавлении новых пользователей, или в процессе авторизации, когда пользователи логинятся в систему. Например, администратор может иметь роль "администратор", а обычный пользователь — роль "пользователь".
- **Шаг 3: Авторизация на основе ролей.** После того как пользователю назначена роль, нужно реализовать логику авторизации. Она будет проверять роль пользователя и определять, может ли он получить доступ к защищенным маршрутам или выполнить определенные действия. Например, администратор может иметь доступ ко всем функциям, а обычный пользователь — только к части функционала (например, к чтению данных или обновлению их).

Обработка доступа на основе ролей

FastAPI предлагает несколько способов управления доступом на основе ролей:

- **Внедрение зависимостей.** **FastAPI** позволяет создавать пользовательские зависимости, которые могут проверять роль пользователя перед предоставлением доступа к определенным конечным точкам. Это позволяет гибко управлять доступом и применять логику проверки ролей в нужных местах приложения.
- **Авторизация на основе декоратора.** Вы можете использовать декораторы для определения авторизации на основе ролей для конкретных конечных точек. Это упрощает процесс управления доступом и позволяет легко настраивать роли, которым разрешен доступ к тем или иным ресурсам.
- **Интеграция с базой данных.** Для более масштабных приложений стоит интегрировать систему **RBAC** с базой данных, где будут храниться роли пользователей и связанные с ними

разрешения. Это позволяет динамически управлять доступом в зависимости от изменений в базе данных.

Реализация управления доступом на основе ролей (RBAC) в FastAPI для начинающих

На данном занятии мы создадим простую систему контроля доступа с ролями `"admin"` и `"user"` с использованием **FastAPI**. Администраторы будут иметь доступ ко всем маршрутам, включая те, что предназначены для обычных пользователей. Мы будем использовать **JWT** для аутентификации и декораторы для авторизации.

Структура проекта

```

.
├── main.py           # Основной файл с FastAPI-приложением
├── security.py       # Функции для работы с JWT и аутентификацией
├── models.py         # Pydantic-схемы для данных
├── db.py             # "База данных" для хранения пользователей
├── rbac.py           # Логика работы с RBAC и декораторы для проверки
ролей
└── dependencies.py   # Общие зависимости, включая получение текущего
пользователя

```

`security.py` – работа с JWT

```

import jwt
import datetime
from fastapi.security import OAuth2PasswordBearer
from fastapi import HTTPException, status, Depends

# Определяем схему аутентификации (OAuth2 с паролем)
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login")

# Секретный ключ для подписи JWT
# В реальном проекте храните его в .env файле, а не в коде!
SECRET_KEY = "mysecretkey" # Генерируем через `openssl rand -hex 32`
ALGORITHM = "HS256" # Используем HMAC SHA-256 для подписи
ACCESS_TOKEN_EXPIRE_MINUTES = 15 # Время жизни токена (15 минут)

def create_jwt_token(data: dict):
    """Создаём JWT-токен с указанием времени истечения"""
    to_encode = data.copy()
    expire = datetime.datetime.utcnow() +
datetime.timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire}) # Добавляем время истечения в токен
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def get_user_from_token(token: str = Depends(oauth2_scheme)):
    """Получаем информацию о пользователе из токена"""

```

```

try:
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM]) #
Декодируем токен
    return payload.get("sub") # JWT-токен содержит `sub` (subject) –
имя пользователя
except jwt.ExpiredSignatureError:
    raise HTTPException(status_code=401, detail="Токен устарел") #
Токен просрочен
except jwt.InvalidTokenError:
    raise HTTPException(status_code=401, detail="Ошибка авторизации")
# Невалидный токен

```

models.py – модели данных Pydantic

```

from pydantic import BaseModel, EmailStr

class User(BaseModel):
    """Модель пользователя с базовыми полями"""
    username: str
    full_name: str | None = None
    email: EmailStr | None = None
    disabled: bool = False
    roles: list[str] # Список ролей пользователя

class UserLogin(BaseModel):
    """Модель для входа в систему"""
    username: str
    password: str

```

db.py – имитация базы данных

```

from models import User

# Фиктивные данные пользователей (в реальном проекте тут будет БД)
USERS_DATA = [
    {
        "username": "admin",
        "password": "adminpass", # В продакшене пароли должны быть
хешированы!
        "roles": ["admin"],
        "full_name": "Admin User",
        "email": "admin@example.com",
        "disabled": False
    },
    {
        "username": "user",
        "password": "userpass",
        "roles": ["user"],

```

```

        "full_name": "Regular User",
        "email": "user@example.com",
        "disabled": False
    },
]

def get_user(username: str) -> User:
    """Получаем пользователя по имени (без пароля)"""
    for user_data in USERS_DATA:
        if user_data["username"] == username:
            return User(**{k: v for k, v in user_data.items() if k !=
"password"})
    return None

```

rbac.py – проверка прав доступа

```

from fastapi import HTTPException, status
from functools import wraps

class PermissionChecker:
    """Декоратор для проверки ролей пользователя"""
    def __init__(self, roles: list[str]):
        self.roles = roles # Список разрешённых ролей

    def __call__(self, func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            user = kwargs.get("current_user") # Получаем текущего
пользователя
            if not user:
                raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Требуется аутентификация")

            if "admin" in user.roles: # Админ всегда имеет доступ ко
всему
                return await func(*args, **kwargs)

            if not any(role in user.roles for role in self.roles):
                raise HTTPException(
                    status_code=status.HTTP_403_FORBIDDEN,
                    detail="Недостаточно прав для доступа"
                )
            return await func(*args, **kwargs)
        return wrapper

```

dependencies.py – вспомогательные функции

```

from fastapi import Depends, HTTPException, status
from security import get_user_from_token

```

```

from db import get_user
from models import User

def get_current_user(current_username: str = Depends(get_user_from_token))
-> User:
    """Получаем текущего пользователя по имени из токена"""
    user = get_user(current_username)
    if user is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="User not found")
    return user

```

main.py – основное приложение

```

from fastapi import FastAPI, Depends, HTTPException, status
from security import create_jwt_token
from models import UserLogin, User
from db import USERS_DATA
from dependencies import get_current_user
from rbac import PermissionChecker

app = FastAPI()

@app.post("/login")
async def login(user_in: UserLogin):
    """Маршрут для аутентификации"""
    for user in USERS_DATA:
        if user["username"] == user_in.username and user["password"] ==
user_in.password:
            # Генерируем JWT-токен для пользователя
            token = create_jwt_token({"sub": user_in.username})
            return {"access_token": token, "token_type": "bearer"}
    raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Неверные учетные данные")

@app.get("/admin")
@PermissionChecker(["admin"])
async def admin_info(current_user: User = Depends(get_current_user)):
    """Маршрут для администраторов"""
    return {"message": f"Hello, {current_user.username}! Welcome to the
admin page."}

@app.get("/user")
@PermissionChecker(["user"])
async def user_info(current_user: User = Depends(get_current_user)):
    """Маршрут для пользователей"""
    return {"message": f"Hello, {current_user.username}! Welcome to the
user page."}

@app.get("/about_me")
async def about_me(current_user: User = Depends(get_current_user)):

```

```
"""Информация о текущем пользователе"""  
return current_user
```

Дополнительные рекомендации

- **Хранение паролей:** Всегда используйте хэширование (например, библиотеку `Passlib`).
- **Минимальная длина пароля:** Не менее `12` символов с комбинацией букв, цифр и специальных символов.
- **Реальная база данных:** Для продакшена замените `USERS_DATA` на подключение к БД (PostgreSQL, MySQL и т. д.).

Тестирование

Авторизация администратора:

```
curl -X POST http://localhost:8000/login -H "Content-Type:  
application/json" -d '{"username":"admin","password":"adminpass"}
```

Доступ к защищённым эндпоинтам:

```
# Для администратора  
curl -H "Authorization: Bearer {TOKEN}" http://localhost:8000/admin  
  
# Для пользователя  
curl -H "Authorization: Bearer {TOKEN}" http://localhost:8000/user
```

Теперь у вас есть базовое понимание, как реализовать **RBAC** в **FastAPI**, и вы можете адаптировать эту систему под свои нужды.

Made with ❤️ by **dvOretsky**